WonderBot: A Comprehensive Briefing on Local Multimodal AI Architecture

Executive Summary

WonderBot represents a shift in artificial intelligence design, moving away from isolated Large Language Model (LLM) instances toward a "small nervous system" architecture. Unlike standard chatbots that operate on a "wait-for-prompt" basis, WonderBot is a multimodal agent built on a continuously ticking substrate. It integrates live sensory inputs—primarily through a remote desktop bridge—with a sophisticated memory lifecycle that includes sleep and dream cycles for long-term information consolidation.The system is optimized for local execution on specialized hardware, specifically dual NVIDIA Tesla P40 GPUs, utilizing a modular backend that supports both lightweight associative memory (LVTC) and high-fidelity LLMs (Qwen). Key innovations include an unbundled architectural stack, a transition toward "tokenizerless" event-coded representations, and a focus on salience-driven spontaneous thought rather than reactive prompting.

I. Architectural Foundation

The architecture of WonderBot is defined by the "unbundling" of three historically conflated problems: representation, continuous agency, and backend generation. The system is structured into five distinct layers to create a living internal state.

1. The Five-Layer Stack

Layer,Component,Functionality
1,Event Codec,"Replaces traditional tokenization with resonant segmentation, lossless byte IDs for recovery, and feature vectors for search."
2,Memory,"An append-only, non-destructive store. Consolidation archives low-priority items while protecting core identity elements."
3,Ganglion / CA Bus,A continuously ticking substrate where event signatures are written as compact patches. The bus updates and bleeds between channels every tick.
4,Resonance Field,Acts as an internal pressure signal for ranking and gating rather than simple naming.
5,Backend,"Downstream processing. Options include Echo (guaranteed local), HF (Hugging Face renderer), and Native (event-stream model)."

2. Design Philosophy

The system prioritizes a "living internal state" that is not reducible to simple API calls. By unbundling the layers, the architecture preserves a path to a truly native event-coded or tokenizerless model while remaining functional on current pre-trained weights.

II. Cognitive Lifecycle and Memory Management

WonderBot employs a sophisticated memory lifecycle (Phases 7 and 8) to distinguish between recent events and durable knowledge.

1. Memory Stores and Lifecycle

- **Long-Term Memory (LTM):** Stored in state/long_term_memory.json, keeping durable entries with metadata such as "strength," "use count," and "evidence."
- **Sleep Cycle (/sleep):** A pass that promotes strong journal entries to LTM while decaying and archiving weak entries that no longer justify active status.
- **Dream Cycle (/dream):** Creates guarded synthetic links between adjacent LTM entries, resembling associative recombination/rehearsal rather than hallucination.

2. The Self-Model and Goal Layer

Phase 8 introduces an identity layer that weights retrieval based on active objectives.

- **Self-Model:** Located in state/self_model.json, it tracks identity cues, preferences, and constraints.
- **Goal Store:** Located in state/goals.json, it manages persistent goals, subtasks, and blockers.
- **Goal-Weighted Recall:** Memory retrieval is influenced by what the agent is actively trying to accomplish, ensuring contextually relevant responses.

III. Sensory Perception and the Remote Bridge

WonderBot utilizes a multimodal input layer that allows it to see and hear its environment, often across a network.

1. Remote AV Bridge

The "Remote Bridge" architecture allows the Linux server (the brain) to remain headless in a rack while a Windows desktop (the senses) streams camera frames and microphone audio over the local network.

- **Client Side (Windows):** Captures hardware-level AV and streams to the server.
- **Server Side (Linux):** Treats remote streams as native sensory inputs.

2. Sensory Processing Mechanisms

The system uses salience gating to determine what observations warrant attention.

- **Visual:** Detects motion, scene changes, and lighting shifts via a camera.
- **Audio:** Uses a Voice Activity Detection (VAD) front-end to distinguish between ambient noise and speech.
- **Transcription:** Utilizes Whisper-family models (e.g., distil-whisper/distil-small.en) for Speech-to-Text (STT).

IV. Hardware Infrastructure and Optimization

The system is specifically tuned for a dual NVIDIA Tesla P40 server environment, requiring precise configuration for stability and performance.

1. GPU Configuration (Tesla P40)

The P40 GPUs are Pascal-architecture cards that require specific handling:

- **Driver Support:** Pinned to the nvidia-driver-570-server or 580-server branch.
- **Persistence Mode:** Must be enabled to prevent latency and bus-drop issues.
- **Power Capping:** Capped at 200W–230W per card to manage thermals in passive-cooling environments.

- **NUMA Pinning:** Requires CPU-side work to be pinned to the GPU's local socket to avoid cross-socket UPI latency.

2. Monitoring and Health

A comprehensive monitoring stack is deployed via Docker Compose, including:

- **Prometheus:** Scrapes metrics every 5 seconds.
- **Grafana:** Provides real-time dashboards for GPU utilization, VRAM usage, temperature, and power draw.
- **Exporters:** node_exporter for host metrics and nvidia_gpu_exporter for specialized GPU data.

V. Operational Commands and Diagnostics

The system provides a suite of CLI tools for manual and autonomous operation.| Command | Category | Purpose || ------ | ------ | ------ || /diagnostics | System | Checks status of camera, mic, STT, and TTS systems. || /sensors | Sensory | Polls remote sensors for a deterministic summary. || /sense | Sensory | Gathers observations and asks the LLM to summarize them. || /watch n | Autonomous | Engages the autonomous sensor loop for a set number of ticks. || /sleep | Cognitive | Runs the memory promotion and decay pass. || /goals | Cognitive | Inspects the current goal store and active work queue. || /remember query | Retrieval | Searches long-term memory for specific content. |

VI. Implementation Challenges and Resolutions

Development and deployment have highlighted critical technical constraints:

- **Tokenizer Limitations:** Traditional tokenizers are identified as a fundamental flaw; the project is moving toward a "resonant tokenizer" that uses lossy/semantic modes for memory chunking.
- **Environment Management:** Heavy AI dependencies (Torch, Transformers) require careful management of HF_HOME and TMPDIR to avoid filling the root partition. The system is designed to offload weights and caches to a dedicated 1.92TB SSD.
- **VAD/STT Tuning:** Early versions struggled with "early cutoff" in speech. The current fix involves a software preamp/AGC (Auto Gain Control) stage and silence-based endpointing to ensure full utterances are captured.
- **Containerization:** The project supports a Docker-based deployment to ensure the environment (CUDA, PyTorch) remains consistent across different host configurations.